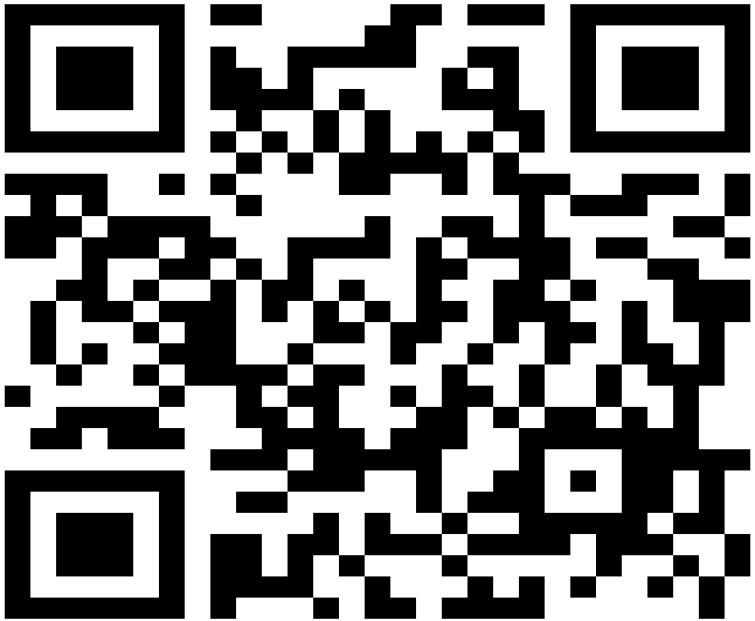
Explore | Expand | Enrich

<Codemithra />™

MERGE SORT N DLL

TEST TIME ON SEGREGATE EVEN AND ODD NODES IN A LINKED LIST

URL:https://forms.gle/s4Wicp5kj3zKiLLX7



MERGE SORT IN DLL

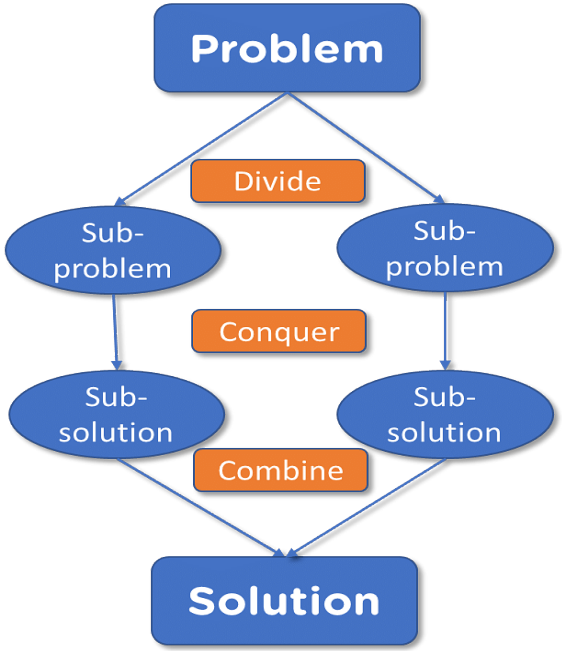
TIME TO RECALL…

- Merge Sort is a widely used and efficient comparison-based sorting algorithm that falls under the category of divide and conquer algorithms.
- It was invented by John von Neumann in 1945. Merge Sort divides an unsorted list into smaller sublists, recursively sorts those sublists, and then merges them back together to produce a sorted list. Here's how it works:

1. Divide 2.Conquer 3. Merge

MERGE SORT IN DLL

TIME TO RECALL…



MERGE SORT IN DLL

- Divide: The unsorted list is divided into two equal halves (or approximately equal if the list has an odd number of elements). This is done recursively for each sublist until each sublist contains only one element.
- Conquer: The individual elements, which are essentially sorted sublists of size one, are considered sorted.
- Merge: The sorted sublists are then merged together in pairs to create larger sorted sublists. This process continues until a single, fully sorted list is achieved.
  - The key step in Merge Sort is the merging phase, where two sorted sublists are combined into a larger sorted sublist. This merging process is efficient because it takes advantage of the fact that both sublists are already sorted.

MERGE SORT IN DLL

ALGORITHM

1. If the linked list is empty or has only one node, it is already sorted, so return it.
2. Find the middle node of the linked list using the slow and fast pointer approach.
3. Split the linked list into two halves at the middle node.
4. Recursively apply Merge Sort to the two halves.
5. Merge the two sorted halves together, preserving the order.
6. Return the merged list as the result

MERGE SORT IN DLL

PSEUDOCODE

```
Function mergeSortDoublyLinkedList(list):
    if list is empty or has only one node:
        return list
    // Find the middle node
    middle = findMiddleNode(list)
    // Split the list into two halves
    leftHalf = sublist from start to middle
    rightHalf = sublist from middle.next to end
```

MERGE SORT IN DLL

PSEUDOCODE

```
// Recursively apply mergeSort to both halves
    leftHalf = mergeSortDoublyLinkedList(leftHalf)
    rightHalf = mergeSortDoublyLinkedList(rightHalf
    // Merge the two sorted halves
    sortedList = merge(leftHalf, rightHalf)
    return sortedList
Function merge(leftHalf, rightHalf):
    result = Initialize an empty doubly linked list
    while leftHalf is not empty and rightHalf is not
empty:
```

MERGE SORT IN DLL

PSEUDOCODE

```
if leftHalf.data < rightHalf.data:
            append leftHalf to result
            move leftHalf pointer to the next node
        else:
            append rightHalf to result
            move rightHalf pointer to the next node
    // Append any remaining nodes from leftHalf and rightHalf (if any)
    while leftHalf is not empty:
        append leftHalf to result
        move leftHalf pointer to the next node
```

MERGE SORT IN DLL

PSEUDOCODE

```
while rightHalf is not empty:
        append rightHalf to result
        move rightHalf pointer to the next node
    return result
```

MERGE SORT IN DLL

Explore | Expand | Enrich

EXAMPLE

Input:

2 4 1 3 5

Output:

1 2 3 4 5

MERGE SORT IN DLL

```
import java.util.*;
//MERGE FUNCTION
class Solution {
    public static Node merge(Node head1, Node
head2){
        Node merged = new Node(-1);
        Node temp = merged;
   while (head1 != null && head2 != null) {
            if (head1.data < head2.data) {
                temp.next = head1;
                if(temp.data != -1)
                    head1.prev = temp;
                head1 = head1.next;
            }else {
                temp.next = head2;
                if(temp.data != -1)
                    head2.prev = temp;
                head2 = head2.next;

}
```

```
temp = temp.next;
        }
        while (head1 != null) {
            temp.next = head1;
            head1.prev = temp;
            head1 = head1.next;
            temp = temp.next;
        }
        while (head2 != null) {
            temp.next = head2;
            head2.prev = temp;
            head2 = head2.next;
            temp = temp.next;
        }
        return merged.next;
    }
```

MERGE SORT IN DLL

```
//2.FIND THE MID POINT

public static Node find_mid(Node head){
        Node slow = head, fast =
head.next;
        while(slow != null && fast !=
null){
            fast = fast.next;
            if(fast == null)
                break;
            slow = slow.next;
            fast = fast.next;
        }
        return slow;
    }
```

```
 public static Node mergesort(Node
head){
        if(head.next == null){
            return head;
        }
Node mid = find_mid(head);
        Node head1 = head;
        Node head2 = mid.next;
        mid.next = null;
        head2.prev = null;
        head1 = mergesort(head1);
        head2 = mergesort(head2);
        return merge(head1, head2);
    }
}
```

MERGE SORT IN DLL

```
//3.NODE
class Node{
    int data;
    Node next;
    Node prev;
    Node(int data){
        this.data = data;
        next = null;
        prev = null;
    }
}
//4.LINKED LIST
class LinkedList{
    Node head;
```

```
void add(int data){
        Node new_node = new Node(data);
        if(head == null){
            head = new_node;
            return;
        }
        Node curr = head;
        while(curr.next != null)
            curr = curr.next;
        curr.next = new_node;
        new_node.prev = curr;
    }
}
```

MERGE SORT IN DLL

```
//5.MAIN FUNCTION
public class Main {
    public static void main(String[] args) {

Scanner input = new Scanner(System.in);
        int n = input.nextInt();
        LinkedList a = new LinkedList();
        for(int i = 0; i < n; i++){
            a.add(input.nextInt());
        }
        Solution Obj = new Solution();
        a.head = Obj.mergesort(a.head);
        Node h = a.head;
        while(h != null){
            System.out.print(h.data + " ");
            h = h.next;
        }
        System.out.println("");
    }
}
```

MERGE SORT IN DLL

```
public class DLLMergeSort {
    private Node head;

    public DLLMergeSort() {
        this.head = null;
    }
    public class Node {
    int data;
    Node prev, next;

    public Node(int data) {
        this.data = data;
        this.prev = this.next = null;
    }
}
```

```
private Node merge(Node left, Node right) {
 if (left == null) {
        return right;   }
 if (right == null) {
        return left;}
 if (left.data < right.data) {
       left.next = merge(left.next, right);
        if (left.next != null) {
            left.next.prev = left;
        }
        left.prev = null;
        return left;
    } else {
     right.next = merge(left, right.next);
        if (right.next != null) {
            right.next.prev = right;
        }
        right.prev = null;
       return right;}}
```

MERGE SORT IN DLL

```
private Node getMiddle(Node head) {
    if (head == null) {
        return null;
    }

    Node slow = head;
    Node fast = head.next;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }

    return slow;
}
```

```
public Node mergeSort(Node node) {
        if (node == null || node.next == null) {
            return node;
        }
        Node middle = getMiddle(node);
        Node nextOfMiddle = middle.next;
        middle.next = null;
        nextOfMiddle.prev = null;
     Node left = mergeSort(node);
     Node right = mergeSort(nextOfMiddle);
     return merge(left, right);
    }
```

MERGE SORT IN DLL

```
public void printList(Node node) {
        while (node != null) {
            System.out.print(node.data + " ");
            node = node.next;
        }
        System.out.println();
    }
```

```
public void insert(int data) {
        Node newNode = new Node(data);
    if (head == null) {
            head = newNode;
        } else {
            Node temp = head;
            while (temp.next != null) {
                temp = temp.next;
            }
            temp.next = newNode;
            newNode.prev = temp;
        }
    }
```

MERGE SORT IN DLL

```
public static void main(String[] args) {
        DLLMergeSort dll = new DLLMergeSort();
        dll.insert(12);
        dll.insert(11);
        dll.insert(13);
        dll.insert(5);
        dll.insert(6);
        dll.insert(7);
        System.out.println("Original DLL:");
        dll.printList(dll.head);
        dll.head = dll.mergeSort(dll.head);
        System.out.println("DLL after Merge Sort:");
        dll.printList(dll.head);
    }
}
```

MERGE SORT IN DLL

Time complexity and space complexity

Time complexity of O(n * log(n)) arises from the Merge Sort algorithm applied to the linked list.

Space complexity of O(log(n)) comes from the recursive function calls on the call stack during the sorting process.

INTERVIEW QUESTIONS

1. Explain how Merge Sort works for sorting a doubly linked list.

Answer: Merge Sort for a doubly linked list involves dividing the list into two halves, recursively sorting both halves, and then merging them back together in sorted order. It takes advantage of the doubly linked list's ability to traverse in both directions.

2.What is the time complexity of Merge Sort when applied to a doubly linked list?

Answer: The time complexity of Merge Sort on a doubly linked list is O(n log n), where n is the number of nodes in the list. This is because the list is divided in a balanced manner, and merging two sorted halves takes linear time.

3. Can Merge Sort be implemented in-place for a doubly linked list?

Answer: Yes, Merge Sort can be implemented in-place for a doubly linked list. It involves reorganizing the pointers of the existing nodes without using additional memory for new nodes.

4. How do you handle the merging step in Merge Sort for a doubly linked list?

Answer:The merging step involves comparing elements of the two sorted halves and rearranging the pointers to form a single sorted list. This is done while traversing both halves and updating the pointers accordingly, similar to merging two sorted arrays.

https://learn.codemithra.com

Explore | Expand | Enrich

THANK YOU

+91 78150 95095 | codemithra@ethnus.com | www.codemithra.com